LAB: GPIO 7-segment(eval board)

LAB: GPIO 7-segment(eval board)

LAB: GPIO 7-segment

Date: 2026-09-02

Author/Partner: Choi Junghyun

Demo Video: https://youtu.be/CbpHnDxY32g

PDF version:

Introduction

In this lab, you are required to create a simple program to control a 7-segment display to show a decimal number (0~9) that increases by pressing a push-button.

You must submit

- LAB Report (*.pdf)
- Zip source files(lab***.c, ecRCC2.h, ecGPIO2.h etc...).
    - Only the source files. Do not submit project files

Requirement

Hardware

- MCU
    - NUCLEO-F411RE
- Actuator/Sensor/Others:
    - eval board

Software

- PlatformIO, CMSIS, EC_HAL library

Exercise

Fill in the table

Port/Pin	Description	Register setting
Port B Pin 5	Clear Pin5 mode	GPIOB->MODER &=~(3<<(5*2))
Port B Pin 5	Set Pin5 mode = Output	GPIOB->MODER |=(1 << (5*2))
Port B Pin 6	Clear Pin6 mode	GPIOB->MODER &=~ (3 << (6*2))
Port B Pin 6	Set Pin6 mode = Output	GPIOB->MODER |=(1 << (6*2))

Port/Pin	Description	Register setting
Port B Pin Y	Clear PinY mode	GPIOB->MODER &=~ (3 << (Y*2))
Port B Pin Y	Set PinY mode = Output	GPIOB->MODER |=(1 << (Y*2))
Port B Pin 5~9	Clear Pin5~9 mode	GPIOB->MODER &=~(0x3FF << (5*2))
	Set Pin5~9 mode = Output	GPIOB->MODER |=(0x155 << (5*2))
Port X Pin Y	Clear Pin Y mode	GPIOX->MODER &=~(3 << (Y*2))
	Set Pin Y mode = Output	GPIOX->MODER |=(1 << (Y*2))
Port B Pin5	Set Pin5 otype=push-pull	GPIOB->OTYPER =~(1 << 5)
Port B PinY	Set PinY otype=push-pull	GPIOB-> OTYPER =~ (1 << Y)
Port B Pin5	Set Pin5 ospeed=Fast	GPIOB->OSPEEDR =(2 << (5*2))
Port B PinY	Set PinY ospeed=Fast	GPIOB-> OSPEEDR =(2 << (Y*2))
Port B Pin 5	Set Pin5 PUPD=no pullup/down	GPIOB->OTYPER =(3 << (5*2))
Port B Pin Y	Set PinY PUPD=no pullup/down	GPIOB-> OTYPER =(3 << (Y*2))

Prelab: 7-segment Display

Procedure

Complete the Tutorial: **7-segment Display : Option 2. Without a 7-segment decoder**

{% embed url="https://ykkim.gitbook.io/ec/ec-course/tutorial/tutorial-7segment-display#option-3.-without-using-a-7-segment-decoder-on-jkit-evaluation-board" %}

You must check the 7-segment display can show all the number from 0 to 9.

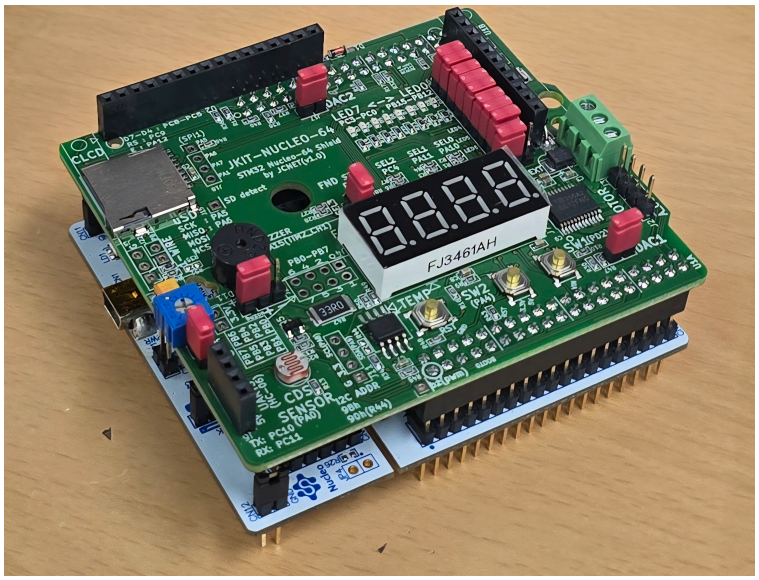
- Give all 'HIGH' and 'LOW' signal to pin 'a'~'g'
- Observe all LEDs are turned ON or OFF
- You must show the result to TA

Problem 1. Library for 7-segment display on JKIT Board

Problem

There are four 7-Segment Displays on JKIT board.

- You need to select which one to use.
- There is no BCD decoder
- JKIT - Nucleo 64: link



Complete the required functions that displays numbers on 7-segment FND.(JKIT - Nucleo 64)

These functions are defined and declared in `ecGPIO2.h,ecGPIO2.c`

```
// Display a number 0 - 9 only
void FND_display_init(PinName_t *pinFND);
void FND_display(uint8_t  num)

// Select display: FND0 to FND3
void FND_select_init(PinName_t *selectFND);
void FND_select(uint8_t select);
```

Display a decimal number: 0~9 on each and all FNDs

{% hint style="info" %} If you want to display multiple 7-segment displays at the same time, you need to use a very short delay to select multiple numbers iteratively {% endhint %}

{% hint style="info" %} **Check if the 7-segments display are Common Anode or Cathode**

For Common Cathode: Giving 'High' to the pin -> LED ON {% endhint %}

Procedure

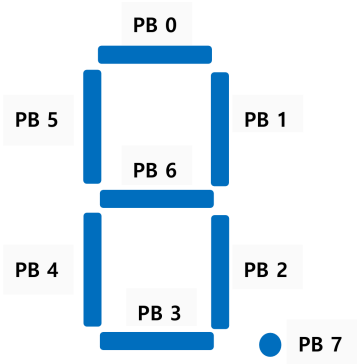
1. Connect the evaluation board (JKIT-NUCLEO) to the MCU.
2. Make sure that your library **ecGPIO2.h, ecGPIO2.c** are in `EC\include\`.
3. Create a new project under the directory `EC\lab\`

- Environment: “**LAB_GPIO_7segment”**
- Source File: ““**LAB_GPIO_7segment.c”**

4. You must modify the `platformio.ini` , to add new environment.

> You MUST write your name in the top of the source file, inside the comment section.

Configuration





Function	Port - Pin	Configuration
Selection of 7-Segment Display	PA_10, PA_11, PC_4, PC_3 (FND_0~FND_3)	DOUT, Push-Pull,
7-Segment LEDs	PB_0, PB_1, PB_2, PB_3, PB_4, PB_5, PB_6, PB_7 ('a'~'g''dot', respectively)	DOUT, Push-Pull, Medium Speed

Example Code

{% tabs %} {% tab title="main_sample" %}

```
#include "stm32f4xx.h"
#include "ecGPIO2.h"
#include "ecRCC2.h"

PinName_t pinFND[8]    = {PB_0, PB_1, PB_2, PB_3, PB_4, PB_5, PB_6, PB_7};
// PinName_t selectFND[4]={PA_10, ... }
// [YOUR CODE GOES HERE]

void setup(void){
    // Intialize System Clock
    RCC_HSI_init();
    // Intialize FND pins and Others
    FND_init(pinFND);
    FND_select_init(selectFND);
    // Intialize  Others
    // [YOUR CODE GOES HERE]
};

int main(void) {
    setup();
    uint8_t numDisplay1  = 0;
    uint8_t numDisplay2  = 5;
    uint8_t selectFND1  = 0;
    uint8_t selectFND2  = 1;
```

```
    while (1) {
        FND_select(selectFND1);
        FND_display(numDisplay1);
        delay_ms_HSI(1);
        FND_select(selectFND2);
        FND_display(numDisplay2);
        delay_ms_HSI(1);
    }
}
```

{% endtab %}

{% tab title="GPIO.h" %} {% code expandable="true" %}

```
////////////////////////////////////////////////////////////////////
/* Include the following in GPIO.h */
////////////////////////////////////////////////////////////////////

// FND for JKIT board
extern int numberFND[12][8];
void FND_init(PinName_t *pinFND);
void FND_select_init(PinName_t *selectFND);
void FND_display(uint8_t num);
void FND_select(uint8_t digit);
```

{% endcode %} {% endtab %}

{% tab title="GPIO.c" %}

```
////////////////////////////////////////////////////////////////////
/* Include the following in GPIO.c */
////////////////////////////////////////////////////////////////////

// FND - JKIT board
static PinName_t _fndPins[8];
static PinName_t _fndSel[4];

//Each led that has to light up gets a 1, every other led gets a 0
//its in order  { (a,b,c,d,e,f,g,dp) }
int numberFND[12][8]={
    {1,1,1,1,1,1,0,0},      //zero
    {0,1,1,0,0,0,0,0},      //one
    {1,1,0,1,1,0,1,0},      //two

    // [YOUR CODE GOES HERE]
    // [YOUR CODE GOES HERE]
    // [YOUR CODE GOES HERE]
```

```
    {0,0,0,0,0,0,0,1}          //dot
    {0,0,0,0,0,0,0,0}             //blank
};


// Initialize DOUT pins for 7 segment leds
void FND_init(PinName_t *pinFND){
    for (int i=0; i<8; i++){
        _fndPins[i] = pinFND[i];
        GPIO_init(_fndPins[i], OUTPUT);
    }
}

// Initialize select pins (FND0~FND3) as DOUT
void FND_select_init(PinName_t *selectFND){
    // [YOUR CODE GOES HERE]
    // [YOUR CODE GOES HERE]
}

// Display a number 0 - 9 only
void FND_display(uint8_t num){
    // [YOUR CODE GOES HERE]
    // [YOUR CODE GOES HERE]
}

// Select display: FND0 to FND3
void FND_select(uint8_t digit){
    // [YOUR CODE GOES HERE]
    // [YOUR CODE GOES HERE]
}

////////////////////////////////////////////////////////////////////
```

{% endtab %} {% endtabs %}

main code

```
/*-----------------------------------------------------------------\
@ Embedded Controller by Young-Keun Kim - Handong Global University
Author           : [Choi Junghyun]
Created          : 05-03-2021
Modified         : 09-23-2026
Language/ver     : C++ in VS Code
Description      : LAB GPIO 7segement - Problem 1
/-----------------------------------------------------------------*/
```

```
#include "stm32f4xx.h"
#include "ecGPIO2.h"
#include "ecRCC2.h"

// define FND LED segment pins (a ~ g, dot)
PinName_t pinFND[8] = {PB_0, PB_1, PB_2, PB_3, PB_4, PB_5, PB_6, PB_7};

// define FND digit select pins (FND_0 to FND_3)
PinName_t selectFND[4] = {PA_10, PA_11, PC_4, PC_3};

void setup(void) {
    // use system clock
    RCC_HSI_init();

    // initialize FND segment pins and digit selection pins
    FND_init(pinFND);
    FND_select_init(selectFND);
}

int main(void) {
    setup();

    // digits display part
    uint8_t numDisplay1 = 0;  // display '0'
    uint8_t numDisplay2 = 5;  // display '5'
    uint8_t selectFND1 = 0;   // display on FND_0 (PA_10)
    uint8_t selectFND2 = 1;   // display on FND_1 (PA_11)

    while (1) {
        // display digit on FND0
        FND_select(selectFND1);
        FND_display(numDisplay1);
        delay_ms_HSI(1);      // Short delay to prevent flickering

        // Display FND1
        FND_select(selectFND2);
        FND_display(numDisplay2);
        delay_ms_HSI(1);
    }
}
```

Problem 2: Counter with Button Press

Procedure

Create a code that increases the displayed number from 0 to 9 with each button press.

- After the number '9', it should start from '0' again.
- Modify from Problem 1 source code

Configuration

Configure the MCU GPIO

Function	Port - Pin	Configuration
Button (SW2) on JKIT	PA_4	DIN, Pull-Up
7-Segment DOUT	PB_0, PB_1, PB_2, PB_3, PB_4, PB_5, PB_6, PB_7 ('a'~'g''dot', respectively)	Push-Pull, No Pull-up-Pull-down, Medium Speed
Selection of 7-Segment Display	PA_10 (FND_0)	DOUT, Push-Pull,

*** Challenge : Extend the display number from 0 to 19**

Code

Sample Code.

Your code goes here:

> Explain your source code with necessary comments.

```
/*----------------------------------------------------------------\
@ Embedded Controller by Young-Keun Kim - Handong Global University
Author           : [Choi Junghyun]
Created          : 05-03-2021
Modified         : 09-23-2026
Language/ver     : C++ in VS Code
Description      : LAB GPIO 7segement - Problem 2
/----------------------------------------------------------------*/


#include "stm32f4xx.h"
#include "ecGPIO2.h"
#include "ecRCC2.h"

#define BUTTON_PIN PA_4

PinName_t pinFND[8]    = {PB_0, PB_1, PB_2, PB_3, PB_4, PB_5, PB_6, PB_7};
PinName_t selectFND[4] = {PA_10, PA_11, PC_4, PC_3};

void setup(void);

int main(void) {
    setup();

    uint8_t count = 0;
    int btnState = HIGH;
```

```
    int prevBtnState = HIGH;

    while (1) {
        // read button current number
        btnState = GPIO_read(BUTTON_PIN);

        // falling edge detection
        if (prevBtnState == HIGH && btnState == LOW) {
            count = (count + 1) % 20;  // counts 0 ~ 19 using %20
            delay_ms_HSI(50);          // software debouncing delay
        }
        prevBtnState = btnState;

        // get digits
        uint8_t tens  = count / 10;
        uint8_t units = count % 10;

        // multiplex display logic
        // display units on very left(FND0)
        FND_select(0);
        FND_display(units);
        delay_ms_HSI(2);

        // display tens on FND1
        if (count >= 10) {
            FND_select(1);
            FND_display(tens);
            delay_ms_HSI(2);
        } else {
            // turn off FND1 for single digit
            FND_select(1);
            FND_display(11);
            delay_ms_HSI(2);
        }
    }
}

void setup(void) {
    RCC_HSI_init();

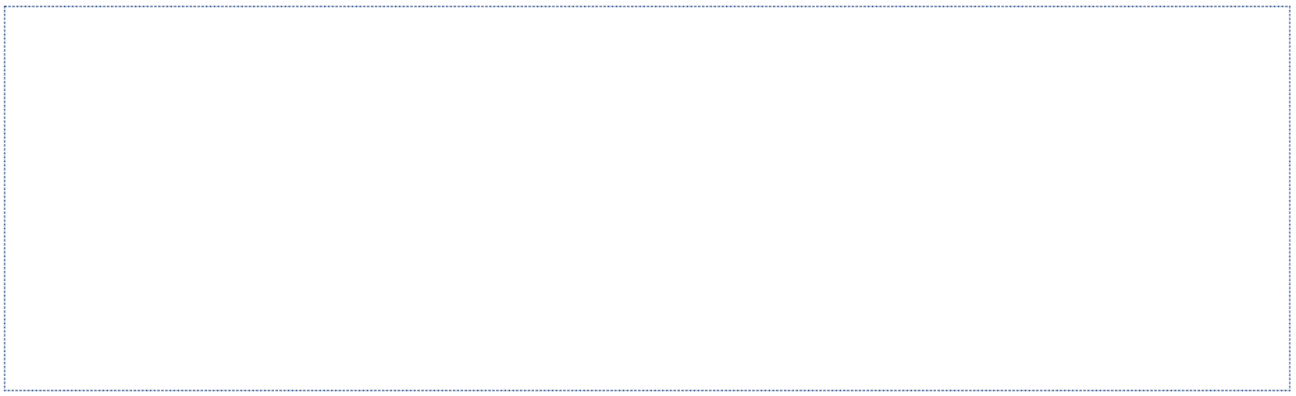
    // SW2 Button is Input with Pull-Up
    GPIO_init(BUTTON_PIN, INPUT);
    GPIO_pupd(BUTTON_PIN, EC_PU);

    // segment pins and select pins
    FND_init(pinFND);
    FND_select_init(selectFND);
}
```

Connection Diagram

Circuit diagram (if needed)

> You need to include the circuit diagram



Results

Experiment images and results

> Show experiment images /results

Add demo video link

Discussion

1. Analyze the result and explain any other necessary discussion.
2. What are the common cathode and common anode of 7-segment display?

> In a 7-segment display, individual LED segments share one common pin. In a common cathode configuration, all the negative terminals of the segment LEDs are connected together to the ground, meaning an individual segment lights up when its corresponding segment pin receives a 1 signal from the MCU. on the other hand, in a common anod configuration, all the positive anodes are connected together to the VCC meaning it turns on when its corresponding pin receives a 0 signal.

3. Does the LED of a 7-segment display (common anode) pin turn ON when 'HIGH' is given to the LED pin from the MCU?

> No, the LED pin will not turn on when given a high(1) signal on a common anode 7-segment display. In a common anode configuration, the positive side of all LEDs are permanently tied to the positive VCC. To turn an individual LED on, the MCU pin connected to the LED cathode must output a 0 signal to ground the circuit and allow current to flow.

4. How can we display 4 digit-numbers on the 4 FNDS at the same time? How to remove ghosting of displaying a number?

> To display a 4-digit number on all 4 7-segment displays simultaneously, a technique called "time-division multiplexing" is used. The MCU rapidly enables one display digit at a time using its selection pin, outputs the corresponding segment data, and holds it for a very brief delay before switching to the next digit. Because this switching occurs faster than the human eye can see, all 4 digits appear continuously illuminated at the same time. for ghosting, to remove it, all segment outputs or selection pins must be turned off right before switching selection pins and applying new segment data. This brief blanking period ensures residual charge and previous data are cleared before the next digit activates.

Reference

Complete list of all references used (github, blog, paper, etc)

```
https://ykkim.gitbook.io/ec/ec-course/example-code#ec_2024-2
```

Troubleshooting

(Option) You can write Troubleshooting section